

LeetCode 1358: Number of Substrings

Target Companies: D.E. Shaw, Amazon

Problem Statement

Given a string s consisting only of characters 'a', 'b', and 'c', return the number of substrings containing at least one occurrence of all these characters.

Key Insight & Logic

Core Concept: Once a sliding window becomes valid (i.e., it contains at least one 'a', 'b', and 'c'), *every larger window extending to the right also automatically becomes valid.*

For example, consider the string prefix "aaabc". Once this window is valid, any additional characters added to the right will continue to form valid windows:

- aaabc (Valid)
- aaabca (Also Valid)
- aaabcab (Also Valid)
- aaabcabc (Also Valid)

If we find a valid window ending at index r , all substrings ending from r up to the end of the string (index $n - 1$) are valid. The number of such substrings is calculated as:

$$\text{ans} += (n - r)$$

Sliding Window Template

```
int l = 0;
int r = 0;

while (r < n) {
    // Include nums[r] in the window

    while (window_is_valid) {
        ans += (n - r);
        l++; // Advance left pointer to shrink the window
    }

    r++; // Advance right pointer to expand the window
}
```

C++ Solution

```
class Solution {
public:
    int numberOfSubstrings(string s) {
        int n = s.size();
        int l = 0;
        int r = 0;
        int ans = 0;
        unordered_map<int, int> freq;

        while (r < n) {
            freq[s[r]]++;

            while (freq.size() == 3) {
                ans += (n - r); // kyunki agar abc valid he to abca abcaa abcaad abo

                freq[s[l]]--;
                if (freq[s[l]] == 0) {
                    freq.erase(s[l]);
                }
                l++;
            }
            r++;
        }

        return ans;
    }
};
```